Recruits PvE Addon — Design Doc

Oct 26, 2026 · @Rafael

Overview

The addon fills the world with AI-run factions that grow their armies from territory, so players face enemies of escalating size as their own strength grows.

- **Platform:** Forge, Minecraft 1.20.1, built as an addon to talhanation's Villager Recruits.
- **Core loop:** POIs produce resources → factions spend resources on soldiers → armies capture more POIs.
- **Pillars:** factions are simulated as data, not live mobs; all POIs are baked into worldgen; generation is deterministic from the world seed.
- **Player experience:** a calm centre at spawn, a war zone of medium powers around it, and larger, stronger empires the farther out the player travels.

Faction types

There are three faction types, and each one originates from its own worldgen structure. Only Empires claim territory.

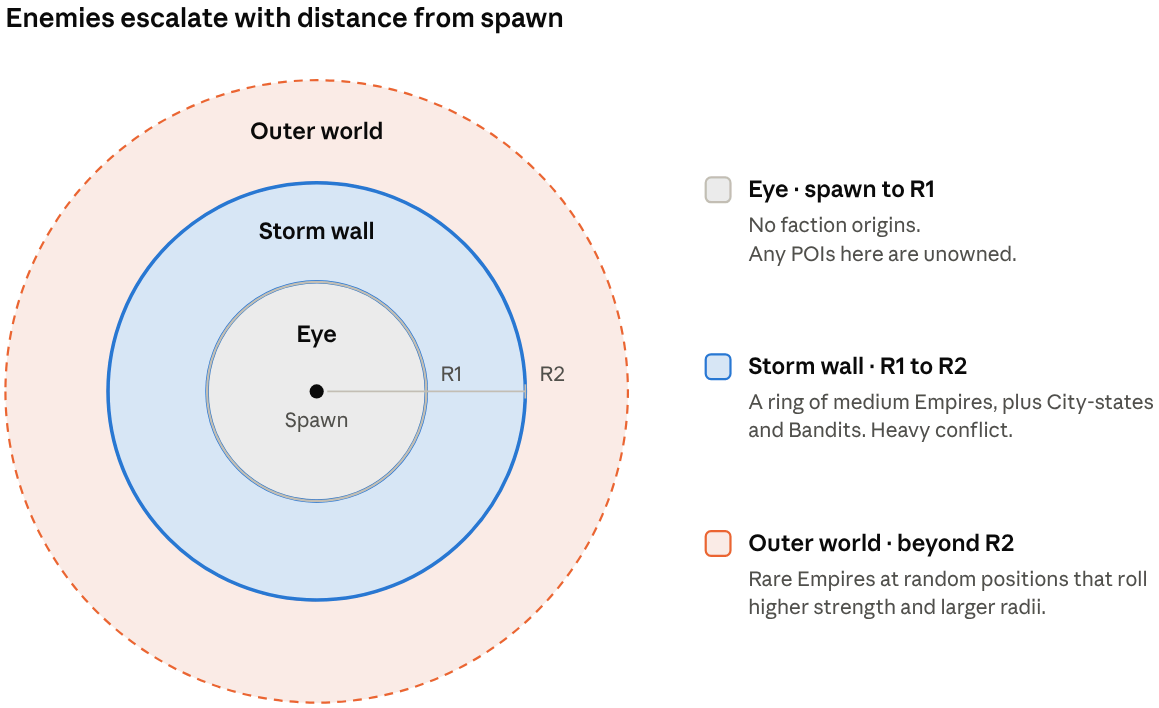
Type	Origin structure	Claims POIs	Growth path
Empire	Capital POI (very rare)	Yes, within its radius	Expands by capturing POIs
City-state	Dedicated City-state structure	No	Grows into an Empire
Bandit	Dedicated Bandit structure	No	Raids; its home becomes a City-state

- **Empire:** each Capital spawns one Empire with a claim radius X of 1–10, where each step is 1,000 blocks.
- **City-state:** holds only its origin structure. Once it passes a strength threshold, it becomes an Empire, gains a radius and starts claiming POIs.

- **Bandit:** holds only its home structure and grows by raiding other factions and players. Once it passes a lower threshold, its home becomes a City-state.
- Faction type is fixed by the origin structure, so it never depends on chunk load order.

World layout

The world is shaped like a hurricane: a calm eye at spawn, a war zone around it, and stronger empires farther out.



hurricane world layout · 3 zones around spawn

- **Eye (spawn → R1):** no faction origins are placed here.
- **Storm wall (R1 → R2):** Capitals are placed in a stronghold-style ring, mixed with City-state and Bandit structures. Empires here are medium-sized.
- **Outer world (beyond R2):** Capitals use random-spread placement with very large spacing and continue as the world expands.
- **Distance scaling:** within the 300–5,000 strength range, Empires roll higher strength and larger radii the farther they are from spawn. Inner Empires might get radius 1–3 and outer ones 6–10.
- R1, R2 and the scaling curves are configurable and frozen per world at creation.

POIs

Every POI is baked into world generation. When a chunk containing a POI generates, its structure generates with it.

What a POI is: a structure that produces resources each in-game day, for example a Farm that produces 5 food per day. Capitals are also POIs.

Virtual POIs: structure placement is deterministic from the seed, so POI positions can be computed before their chunks exist, the same way `/locate` works. Factions own and earn from these virtual POIs right away. When a player generates the chunk, the structure appears and links to the existing record.

Ownership at generation:

1. Find every Empire whose radius covers the POI.
2. If one Empire covers it, the POI goes to that Empire.
3. If several cover it, the POI goes to the Empire with the highest strength. If strengths tie, the nearest Capital wins, then a position hash.
4. If none covers it, the POI is unowned and will be fought over.

Ownership is recorded the first time a POI is computed and frozen after that. Only battles change it. Buildings generate neutral, and faction banners and colours are applied on chunk load from the current owner.

Strength and soldier pool

Strength is a faction's soldier pool: 1 strength = 1 recruit soldier available to that faction.

Value	Range
Strength at generation	300–5,000
Strength maximum	400,000

- **In battle,** a faction draws from its soldier pool to spawn real Recruits entities. Soldiers lost in battle are removed from the pool.
- **At generation,** strength skews with distance from spawn while staying inside 300–5,000. Empires near the war zone roll low, and far-out Empires roll high.
- Strength also decides POI ownership where Empire radii overlap.

Economy loop

The simulation advances once per in-game day, entirely as data.

1. **Produce:** each owned POI adds its daily output to its faction's stockpile, for example a Farm adds 5 food.
2. **Upkeep:** armies consume resources.
3. **Decide:** the faction spends what's left by recruiting (stockpile → strength), expanding toward unowned POIs, or attacking.
4. **Resolve:** captures and raids change POI ownership and stockpiles.

Unowned POIs produce nothing until a faction holds them, which gives factions a reason to expand. Bandits skip production and grow from raid loot.

Technical architecture

Faction state lives in world save data, and real entities spawn only near players.

Component	Role
Faction save data	Factions: id, type, stockpiles, strength, radius, origin position
POI save data	POIs: position, type, owner, daily output, generated yes/no
Custom structure placement type	Places origins by zone, ring count, spacing and distance from spawn
Structure data (JSON)	`structure`, `structure_set` and jigsaw `template_pool` per POI type
Daily tick handler	Runs the economy loop on the server thread
Chunk-load hook	Links a generated structure to its POI record and applies faction colours
Recruits bridge	Maps each faction to a Recruits team; spawns recruits from the soldier pool in battle

Determinism rules:

- Origin radius and strength come from a hash of the world seed and the origin's position, never from a random roll at generation time.
- Zone bounds and scaling curves are copied into save data when the world is created. Later config edits apply only to new worlds.
- Computing a POI's owner requires knowing every Capital within 10,000 blocks. Placement math does this without generating chunks.

Threading: chunk generation runs off the main thread. Worldgen code never writes save data; it queues updates back to the server thread.

First milestone

Goal: one Empire earning food from Farms, visible through debug commands.

- [ ] Forge 1.20.1 project (mod ID rafasconquest) with the Villager Recruits dependency
- [ ] Capital and Farm structures with placement rules
- [ ] Virtual POI precompute around spawn
- [ ] Radius and strength ownership assignment
- [ ] Daily food tick
- [ ] Debug commands: `/pve faction info`, `/pve poi list`

Milestone 1 implementation brief

This section is written for Claude Code. Build exactly Milestone 1 and nothing more.

Scope fence. Do not build battles, recruit spawning, captures, raids, City-state or Bandit origins, growth transitions, or any resource besides food. Leave clear extension points for them, but no gameplay.

Project

Setting	Value
Loader	Forge, Minecraft 1.20.1, Forge 47.x, Java 17
Template	Official Forge 1.20.1 MDK (ForgeGradle)
Mod ID	`rafasconquest` (Forge requires lowercase)
Display name	RafasConquest

Setting	Value
Base package	`rafasconquest` (lowercase by Java convention)
Recruits	Newest Villager Recruits release for Forge 1.20.1 on Modrinth; 1.15.0 confirmed, Modrinth version ID `2lCoSzFD`

Recruits dependency. Add the Modrinth Maven repository (`https://api.modrinth.com/maven`, group `maven.modrinth`) and depend on `maven.modrinth:villager-recruits:<version>` through `fg.deobf` so dev runs load it. At setup, check Modrinth for a newer 1.20.1 Forge release than 1.15.0 and use it if present. Declare Recruits as a required dependency in `mods.toml` with version range `[1.15.0,)`. Milestone 1 code must not call the Recruits API; put all future Recruits contact behind one `RecruitsBridge` interface.

Starting values

All values are config entries, copied into world save data when a world is created and read from there afterwards.

Setting	Placeholder
Eye radius R1	1,000 blocks
War-zone outer edge R2	4,000 blocks
Storm-wall Capital ring	6 Capitals at 2,500 blocks from spawn, ±400 jitter, evenly spaced by angle with a seeded offset
Outer-world Capitals	One per 8,000 × 8,000 block grid cell beyond R2, at a seeded position inside the cell
Farm placement	Random spread, spacing 32 chunks, separation 12, plains, savanna, forest and taiga biomes
Farm output	5 food per in-game day
Storm-wall Empire rolls	Radius 1–3, strength 300–1,500

Setting	Placeholder
Far Empire rolls (R2 + 20,000 blocks and beyond)	Radius 6–10, strength 3,500–5,000
Between R2 and R2 + 20,000	Range bounds interpolate linearly with distance

Data model

One `SavedData` attached to the overworld, saved as `rafasconquest_world`.

- `WorldSettings`: the frozen copy of the starting values above, including `int maxCatchUpDays`.
- `Faction`: `UUID id`, `String name`, `FactionType type`, `BlockPos origin`, `int radiusSteps` (1–10), `int strength` (0–400,000), `Map<ResourceType, Integer> stockpile`.
- `Poi`: `BlockPos pos` (unique key), `PoiType type`, `@Nullable UUID owner`, `int dailyOutput`, `boolean generated`.
- Enums: `FactionType { EMPIRE, CITY_STATE, BANDIT }`, `PoiType { CAPITAL, FARM }`, `ResourceType { FOOD }`.
- Clock: `long lastProcessedDay` (the overworld day index, dayTime ÷ 24,000, last processed), `long lastSavedRealTime` (epoch milliseconds, written on every save).
- `Set<Long> precomputedRegions`: keys of the 2,048 × 2,048 block regions already precomputed.

These fields are final for Milestone 1. Every field is written to and read from NBT, and a save → reload round trip must return identical values.

Behaviour

1. **Origin rolls.** A Capital's radius, strength and name come from a hash of the world seed and its position, using the distance scaling above. Never use an unseeded random.
2. **Virtual POIs.** On server start, precompute every Farm and Capital within 8,000 blocks of spawn from the placement rules and the biome source, without generating chunks. After that, precompute lazily in 2,048 × 2,048 block regions as players come within 4,000 blocks of an unprecomputed region.

3. **Ownership.** Assign each Farm once, when first precomputed: among Empires whose radius (radiusSteps × 1,000) covers it, the highest strength wins; ties go to the nearest Capital, then the lower position hash. No covering Empire → unowned. Never reassign in Milestone 1.
4. **Structures.** Register a custom structure type per POI with code-built placeholder pieces: a fenced wheat plot for a Farm and a small stone keep for a Capital. Data files (`structure`, `structure_set`) must use the same placement values as the precompute, so both agree on positions.
5. **Chunk load.** When a chunk with one of these structure starts loads, mark the matching POI `generated`. Worldgen threads never touch save data; queue the update to the server thread.
6. **Daily tick.** The tick follows the overworld clock. Day index = overworld dayTime ÷ 24,000, rounded down; a new day starts each time dayTime crosses a multiple of 24,000 (in-game time 0, sunrise). Each server tick, if the day index is above `lastProcessedDay`, run one production step per day crossed (every owned POI adds its daily output to its owner's stockpile), then set `lastProcessedDay` to the day index. The step always lands at the start of a day, whatever `/time` commands do. Edge cases:
   - **Sleeping** jumps the clock to the next morning, which crosses one boundary: exactly one day is processed.
   - **`/time add`** that crosses several boundaries processes each day crossed.
   - **`/time set`** to a lower value moves the day index backwards: set `lastProcessedDay` to the new index and process nothing, so no day is ever paid twice.
   - **`doDaylightCycle false`** stops the clock, so no days pass.
7. **Offline catch-up (off by default).** Register a boolean gamerule `PassIngameTimeForIRLTime`, default `false`, in the misc category, with this description in the lang file: "In game days are caught up when the server is offline. One hour ~= 3 days worth of resource collection. Off by default."
   - **Off:** no days are added for time the server was offline.
   - **On:** on server start, missed days = (now − `lastSavedRealTime`) ÷ 20 minutes, rounded down (1 hour = 3 days), capped by `maxCatchUpDays` (config, default 0 = no cap). Catch-up runs at most 50 days per server tick, continuing on later ticks until done, so a long backlog never freezes the server.
   - Catch-up only pays resources. It does not move the in-game clock.

    - `lastSavedRealTime` is written on every save even while the gamerule is off, so turning it on later counts only time offline after the last save.
8. **Logging.** Log one info line per catch-up batch (days processed, days remaining) and a summary when catch-up finishes.

Commands (permission level 2)

- `/pve faction list`: every known faction with type, origin, radius and strength.
- `/pve faction info <name|nearest>`: one faction's stats, stockpile and POI count.
- `/pve poi list [radius]`: POIs near the caller with type, owner and generated flag.
- `/pve debug advance <days>`: runs the daily tick that many times.

Acceptance criteria

Acceptance criteria are the checks that decide when Milestone 1 is done. Claude Code should run each one, show the result, and treat any failure as unfinished work. Each check names its steps and the exact result expected.

#	Check	Steps	Pass when
1	Builds	Run `./gradlew build`	Build succeeds with no errors
2	Loads	Run the client dev run with Recruits installed	Game reaches the title screen; log shows RafasConquest and Recruits loaded
3	Storm-wall factions	Create a world, run `/pve faction list`	6 Empires, each 2,100–2,900 blocks from spawn, radius 1–3, strength 300–1,500
4	Determinism	Create a second world with the same seed, run `/pve faction list`	Same names, positions, radii and strengths as check 3
5	Virtual POIs	Before exploring, run `/pve poi list 3000`	Farms listed with owners or "unowned"; all show `generated: false`
6	Ownership rule	Find a Farm covered by two Empires	Owner is the Empire with higher strength

#	Check	Steps	Pass when
7	Structure link	Teleport to a listed Farm	Placeholder farm builds at that spot; `/pve poi list` shows `generated: true`
8	Daily tick	Note an Empire's food, run `/pve debug advance 3`	Food rose by exactly 3 × 5 × that Empire's owned Farms
9	Real tick timing	Note food, then run `/time set 23999` and wait a few seconds	Food rises by one day's output as the clock passes sunrise
10	Time commands	Sleep through a night; then `/time add 72000`; then `/time set 0`	Sleeping pays 1 day; the add pays 3 days; the set pays nothing and no later day is paid twice
11	Offline catch-up	Note food, stop the server, wait 40 minutes, restart. Repeat after `/gamerule PassIngameTimeForIRLTime true`	Gamerule off (default): food unchanged. Gamerule on: log reports 2 caught-up days and food rose by 2 days' output
12	Persistence	Restart the server with no wait	Factions, POIs, owners, stockpiles and day counter unchanged
13	Unit tests	Run `./gradlew test`	Tests pass for ownership and tie-breaks, distance scaling, day counting, catch-up math and NBT round trip

When all 13 pass, Milestone 1 is done. Report the results as a filled-in copy of this table.

Open questions

- [ ] Exact values for the eye radius (R1) and the war-zone outer edge (R2)
- [ ] Strength thresholds for City-state → Empire and Bandit → City-state
- [ ] Radius a City-state receives when it becomes an Empire
- [ ] How a faction captures an unowned POI: first army to arrive, closest faction, or a battle

- [ ] Resource types beyond food, and what each is spent on
- [ ] Daily output per POI type
- [ ] Rarity and spacing for City-state and Bandit structures
- [ ] Battle resolution when no player is nearby